In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
```

In [2]:
```
df = pd.read_csv("city_day.csv")
```

In [3]:
```
df.columns = df.columns.str.strip()
```

In [4]:
```
df = df.ffill()
```

In [5]:
```
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
    df['year'] = df['Date'].dt.year
    df['month'] = df['Date'].dt.month
    df['day'] = df['Date'].dt.day
    df['hour'] = df['Date'].dt.hour
    df = df.drop(['Date'], axis=1)
```

In [6]:
```
if 'PM2.5' in df.columns and 'PM10' in df.columns:
    df['PM_ratio'] = df['PM2.5'] / (df['PM10'] + 1)

if 'NO2' in df.columns and 'CO' in df.columns:
    df['NO2_CO_ratio'] = df['NO2'] / (df['CO'] + 1)
```

In [7]:
```
df = df.select_dtypes(include=np.number)
```

In [8]:
```
df = df.fillna(df.mean())
```

In [9]:
```
X = df.drop('AQI', axis=1)
y = df['AQI']
```

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [11]:
```
lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
```

In [12]:
```
dt = DecisionTreeRegressor(max_depth=10)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
dt_rmse = np.sqrt(mean_squared_error(y_test, dt_pred))
```

In [13]:
```
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
rf_pred = rf.predict(X_test)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))
```

In [14]:
```
cv_scores = cross_val_score(rf, X, y, cv=5, scoring='neg_mean_squared_error')
cv_rmse = np.sqrt(-cv_scores.mean())
```

In [15]:
```
print("Linear Regression RMSE:", lr_rmse)
print("Decision Tree RMSE:", dt_rmse)
print("Random Forest RMSE:", rf_rmse)
print("Cross-Validation RMSE:", cv_rmse)
```

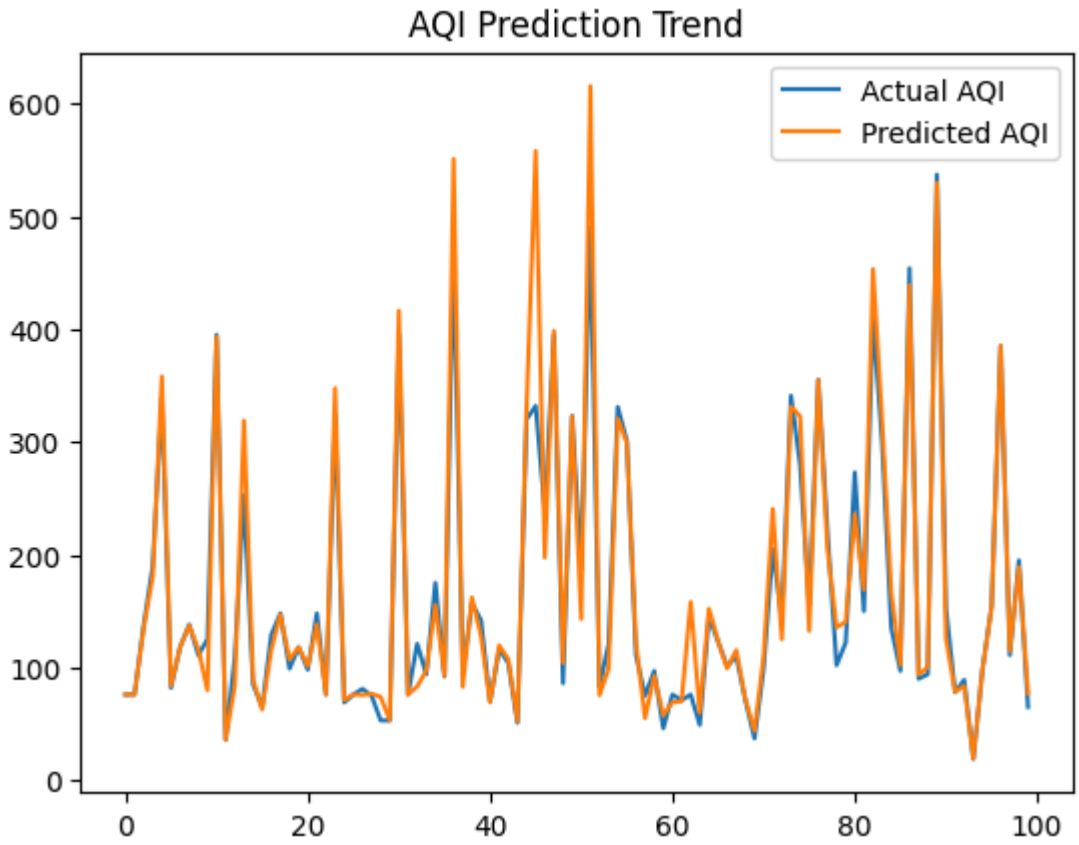
```
Linear Regression RMSE: 58.2373427586922
Decision Tree RMSE: 53.444510764729095
Random Forest RMSE: 40.54953879626157
Cross-Validation RMSE: 90.83049528761636
```

In [16]:
```
predicted_df = pd.DataFrame({
    'Actual AQI': y_test,
    'Predicted AQI': rf_pred
})

print(predicted_df.head())
```

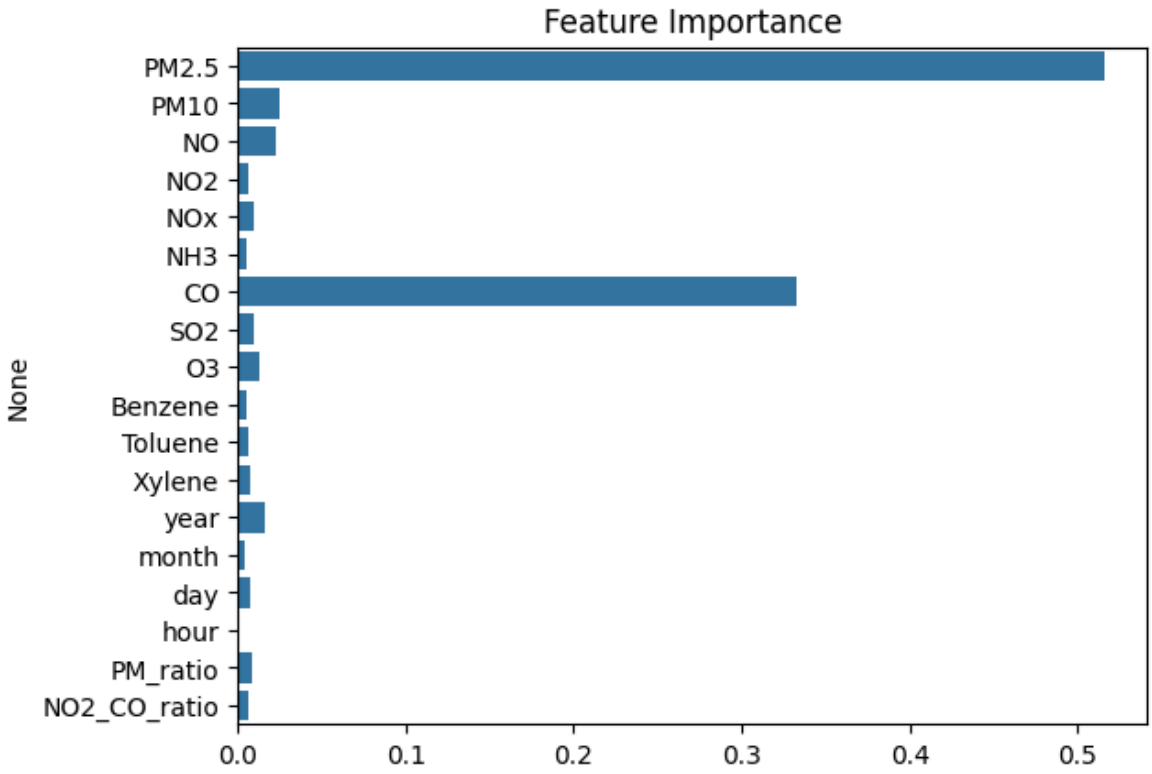
```
       Actual AQI  Predicted AQI
22593        76.0          76.00
22459        76.0          76.00
24213       137.0         134.84
25301       190.0         180.86
20886       339.0         358.11
```

In [17]:
```
plt.figure()
plt.plot(y_test.values[:100], label='Actual AQI')
plt.plot(rf_pred[:100], label='Predicted AQI')
plt.legend()
plt.title("AQI Prediction Trend")
plt.show()
```



In [18]:
```
importance = rf.feature_importances_
features = X.columns

plt.figure()
sns.barplot(x=importance, y=features)
plt.title("Feature Importance")
plt.show()
```



In [19]:
```
def classify_aqi(aqi):
    if aqi <= 50:
        return "Good"
    elif aqi <= 100:
        return "Moderate"
    else:
        return "Poor"

predicted_df['Category'] = predicted_df['Predicted AQI'].apply(classify_aqi)

print(predicted_df.head())
```

```
       Actual AQI  Predicted AQI  Category
22593        76.0          76.00  Moderate
22459        76.0          76.00  Moderate
24213       137.0         134.84      Poor
25301       190.0         180.86      Poor
20886       339.0         358.11      Poor
```

In [ ]: